Mini Project 3: Agentic AI in FinTech Comparing Baseline, Single-Agent, and Multi-Agent Architectures

Eugenio Rivera Ramos
Department of Electrical and Computer Engineering
University of Washington
Seattle, WA, USA
eugenior@uw.edu

Oguz Sinanoglu
Department of Electrical and Computer Engineering
University of Washington
Seattle, WA, USA
oguzs@uw.edu

I. Introduction

This report describes the design, implementation, and evaluation of three increasingly sophisticated AI architectures for answering financial questions using live market data. The project, Mini Project 3 for EE 596, tasks us with building agents that interact with real financial APIs and a local stock database to answer questions ranging from simple ticker lookups to complex cross-domain analyses.

We implement and compare three architectures: (1) a **Baseline** that uses a single LLM call with no tool access, (2) a **Single Agent** that equips one LLM with all seven tools, and (3) a **Multi-Agent** system that routes questions to domain-specific specialists, verifies their evidence, and aggregates the results. All architectures are evaluated on 15 benchmark questions across three difficulty tiers using an LLM-as-a-Judge evaluator, and deployed through a Streamlit chat interface with conversational memory.

II. System Architecture

A. Tool Suite

Seven tools provide live financial data:

1) `get_price_performance` – % price change for tickers over configurable periods (1mo, 3mo, 6mo, ytd, 1y) via `yfinance`.
2) `get_market_status` – open/closed status for global exchanges via Alpha Vantage.
3) `get_top_gainers_losers` – today's top gaining, losing, and most active tickers via Alpha Vantage.
4) `get_news_sentiment` – recent headlines with Bullish/Bearish/Neutral sentiment labels via Alpha Vantage.
5) `query_local_db` – arbitrary SQL `SELECT` on `stocks.db` (ticker, company, sector, industry, market_cap, exchange).
6) `get_company_overview` – fundamentals (P/E, EPS, market cap, 52-week range) via Alpha Vantage with `yfinance` fallback.
7) `get_tickers_by_sector` – sector/industry lookup from the local database with exact-match and fuzzy fallback.

Tools 6 and 7 were implemented as part of Task 1. Tool 6 includes a `yfinance` fallback path to handle Alpha Vantage rate limiting, and Tool 7 performs a case-insensitive exact match on the `sector` column before falling back to a `LIKE` match on `industry`.

B. Baseline

The Baseline uses a single `gpt-4o-mini` (or `gpt-4o`) call with no tools. It relies entirely on the model's training data, which means it cannot retrieve current prices, P/E ratios, or news sentiment. The system prompt instructs the model to answer honestly and acknowledge when data may be outdated.

C. Single Agent

The Single Agent wraps the same model with access to all seven tools and runs in an agentic loop: the model can call tools, inspect results, and call more tools until it produces a final text answer (up to 8 iterations). The system prompt enforces tool-first behaviour: "Always use a tool to retrieve live data before answering."

D. Multi-Agent: Adaptive Router-Verifier

The Multi-Agent system decomposes the problem into four stages, shown in Figure 1:

a) Task Router: A single LLM call classifies the question into up to three domains (market data, fundamentals, news sentiment) and writes a focused sub-question for each active domain. The router also decides the execution mode: *parallel* when sub-tasks are independent, or *staged* when one agent's output is needed as input for another (e.g., "find top semiconductor stocks by return, *then* get their P/E ratios").

b) Specialist Agents: Three specialist agents, each scoped to a subset of tools:

- **MarketAgent** – `get_price_performance`, `get_top_gainers_losers`, `get_market_status`
- **FundamentalsAgent** – `get_company_overview`, `get_tickers_by_sector`, `query_local_db`
- **NewsAgent** – `get_news_sentiment`

Only the agents flagged by the router are activated, avoiding unnecessary API calls on simple questions.

c) Evidence Verifier: After each specialist runs, a rule-based verifier assigns a confidence score in $[0, 1]$. The score starts at 0.5 and is adjusted: $+0.3$ if the agent called at least one tool, $+0.1$ for schema compliance (only allowed tools used), $+0.1$ for a substantive answer (>20 characters), -0.3 for no tool calls (hallucination risk), and -0.1 for reported data gaps. This mechanical check avoids the cost and inconsistency of a second LLM evaluation.

d) Aggregator: Receives all verified specialist results sorted by confidence and produces a single coherent answer. Higher-confidence sources are preferred in case of conflicts.

III. Design Diagram

IV. Implementation Details

A. Technology Stack

- **OpenAI API** (`gpt-4o-mini`, `gpt-4o`) – all LLM calls and tool-calling via the function-calling API.
- **Alpha Vantage** (premium key) – market status, top movers, news sentiment, and company fundamentals.
- **yfinance** – price performance data and fundamentals fallback.
- **SQLite** (`stocks.db`) – local database of S&P 500 companies with sector, industry, market cap, and exchange.
- **Streamlit** – web interface with session-state conversational memory.
- **python-dotenv** – API keys loaded from `.env`.

B. Agentic Loop

All agents share a single `run_specialist_agent()` function that implements the core loop: send the system prompt and task to the LLM, execute any tool calls, append tool results to the conversation, and repeat until the model produces a final text response or the iteration limit (8) is reached. Tool dispatch uses a name-to-function dictionary, and all tool results are serialised as JSON.

C. Multi-Agent Execution Modes

The router's `execution_mode` field controls specialist scheduling:

- **Parallel**: all active specialists run concurrently via `ThreadPoolExecutor`. Used when sub-tasks are independent (most cases).
- **Staged**: the `staged_first` agent runs first, and its answer is appended as context to the remaining agents' tasks before they run in parallel. Used for dependency chains like "find tickers, then get their performance."

D. Conversational Memory (Streamlit)

The Streamlit app maintains a `st.session_state["messages"]` list of all user/assistant turns. Before each agent call, the `build_question_with_history()` function prepends the last 6 messages (3 exchanges) to the current question, enabling agents to resolve follow-up references like "that," "the two," or "it." Long assistant responses are truncated to 800 characters to avoid token bloat.

V. LLM-as-a-Judge Evaluator

A. Scoring Rubric

The evaluator scores each agent answer on a 0–3 scale:

- **3** – Fully correct: satisfies the expected answer description with no invented entities.
- **2** – Mostly correct: minor omission or slight inaccuracy.
- **1** – Partially correct: relevant attempt but missing key parts.
- **0** – Incorrect or refusal.

B. Hallucination Detection

A critical design challenge: the evaluator must distinguish between tool-retrieved precise values (legitimate) and training-data guesses presented as facts (hallucinations). Our evaluator uses two layers:

a) Rule-based pre-check: If the expected answer requires tool retrieval (contains phrases like "from Alpha Vantage") *and* the agent's answer uses hedging language ("approximately," "based on current market conditions," "around"), the answer is immediately flagged as a hallucination with score 1. Refusals are *not* flagged as hallucinations.

b) LLM judge: For all other cases, the evaluator prompts the LLM with a detailed rubric including worked examples for tool-retrieved values (not hallucination), hedged estimates (hallucination), and refusals (not hallucination, score 0).

C. Calibration

Three calibration tests verify correct behaviour: (1) a precise P/E value scores 3 with no hallucination detected, (2) an "approximately" estimate is flagged as hallucination with score ≤ 1, and (3) a refusal scores 0 with no hallucination.

VI. Results

A. Overall Performance (gpt-4o-mini)

Table I reports the per-tier and overall accuracy from `results_gpt4o_mini.xlsx`.

TABLE I: Evaluation results by architecture and difficulty tier (gpt-4o-mini).

Architecture	Easy	Medium	Hard	Overall
Baseline	26.7%	0%	0%	11.1%
Single Agent	80.0%	86.7%	33.3%	75.0%
Multi-Agent	93.3%	86.7%	50.0%	83.3%

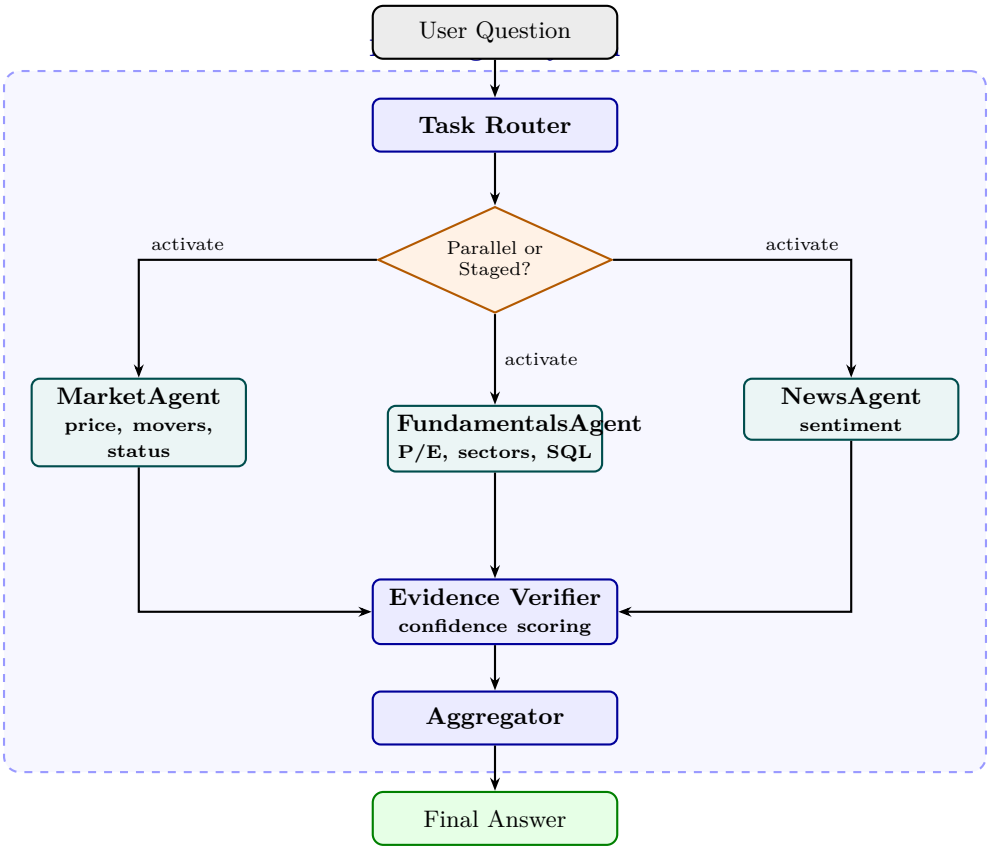

Fig. 1: Design diagram of the Adaptive Router-Verifier multi-agent architecture. The Task Router selects which specialist agents to activate and whether to run them in parallel or staged mode. Each specialist's output is scored by the Evidence Verifier before the Aggregator merges findings into a single answer.

B. *Overall Performance (gpt-4o)*

Table II reports the same metrics with `gpt-4o`.

TABLE II: Evaluation results by architecture and difficulty tier (gpt-4o).

Architecture	Easy	Medium	Hard	Overall
Baseline	26.7%	0%	0%	8.9%
Single Agent	73.3%	86.7%	40.0%	66.7%
Multi-Agent	80.0%	80.0%	40.0%	66.7%

C. *Baseline vs. Single Agent*

The baseline scored 11.1% overall (gpt-4o-mini) because it has no access to live data. On easy questions like Q01 (list semiconductor companies), the baseline earned partial credit by recalling well-known tickers from training data, but on any question requiring a current number (Q03: AAPL P/E ratio, Q05: NVDA 1-month performance), it either refused or guessed. The single agent's 75% accuracy is entirely attributable to tool access: it retrieves live data before answering, eliminating the fundamental information gap.

D. *Single Agent vs. Multi-Agent*

With gpt-4o-mini, Multi-Agent (83.3%) outperformed Single Agent (75%) overall, with the largest gap on easy questions (93.3% vs. 80%) and hard questions (50% vs. 33.3%). On easy questions the improvement comes from cleaner tool usage: the router activates only the relevant specialist, reducing the chance of the model calling an unnecessary tool or getting confused by schema options. On hard questions like Q13 (top 3 semiconductor stocks by return + P/E + sentiment), the specialist decomposition allows each agent to focus on its domain rather than juggling 8+ tool calls in a single context window.

However, Multi-Agent also introduces overhead. On medium questions like Q06 (compare 1-year returns for 3 tickers), Single Agent solves the problem with one `get_price_performance` call. The Multi-Agent system routes it through an orchestrator, spins up a MarketAgent, then runs an aggregator – producing the same answer with 2–3× the latency.

E. *gpt-4o-mini vs. gpt-4o*

Surprisingly, gpt-4o did not consistently outperform gpt-4o-mini. For the Multi-Agent architecture, gpt-4o-mini achieved 83.3% overall vs. gpt-4o's 66.7%. The larger

model was more conservative: on Q11 (tech stocks that dropped this month but grew this year), gpt-4o's Multi-Agent concluded "no verified stocks meet the criteria" rather than returning partial results, while gpt-4o-mini at least identified one qualifying stock. The larger model also had more hallucination flags (3 vs. 2 for Multi-Agent), primarily on Q10 and Q14 where the evaluator flagged values it could not confirm as tool-retrieved.

On easy and medium single-domain questions, both models performed comparably, suggesting that the smaller model is sufficient for straightforward tool-calling tasks and the cost of gpt-4o is not justified for this workload.

F. Hallucination Analysis

The baseline produced zero hallucinations because it consistently refused rather than guessing – the safest failure mode. The Single Agent had 1 hallucination flag (gpt-4o-mini) on Q10 where Goldman Sachs' 52-week values appeared implausible. The Multi-Agent system had 2 hallucinations (gpt-4o-mini), both on fundamentals questions where the aggregator reported values without sufficient evidence of tool retrieval. With gpt-4o, hallucinations increased to 3, partly due to the evaluator's stricter assessment of the larger model's outputs.

G. Latency

Average response times reflect the architectural complexity: Baseline ≈2.6s, Single Agent ≈5.3s, Multi-Agent ≈10.2s (gpt-4o-mini). The Multi-Agent overhead is acceptable for complex cross-domain questions where it adds value, but represents unnecessary cost for simple queries.

VII. Challenges and Design Decisions

a) Router accuracy: The Task Router occasionally misclassifies domain requirements. For Q15 (finance sector stocks near 52-week low + sentiment), the router correctly identified all three domains but the FundamentalsAgent failed to retrieve finance sector tickers because it searched for "Finance" instead of "Financials" (the actual sector name in the database). This cascading failure caused the entire pipeline to return no data.

b) Staged vs. parallel execution: Many hard questions have implicit dependencies: "find the top 3 stocks by X, then get Y for each." The router must detect these dependencies and use staged execution so that the first agent's output (the ticker list) is available to subsequent agents. When the router incorrectly chooses parallel mode, the downstream agents lack the tickers they need and either guess or fail.

c) Evidence Verifier limitations: The rule-based verifier scores confidence mechanically based on tool usage and answer length. It cannot detect *semantic* errors – for example, an agent that calls the correct tool but misinterprets the result. A more sophisticated verifier could cross-check the agent's textual claims against the raw tool output JSON, but this would require an additional LLM call per specialist.

d) Alpha Vantage rate limiting: The premium API key mitigates most rate limits, but concurrent specialist calls in parallel mode can still trigger throttling. Tool 6 (`get_company_overview`) includes a `yfinance` fallback to handle this gracefully.

e) Evaluator bias: The LLM-as-a-Judge tends toward leniency: it sometimes awards a score of 2 to answers that are substantively incomplete. The rule-based pre-check for hedging language helps catch the most egregious cases, but the evaluator still struggles with partial answers that look plausible but omit critical data points.

VIII. Streamlit Deployment

The Streamlit app (`streamlitapp/app.py`) wraps the notebook agents into a chat interface with:

- **Sidebar controls**: Agent selector (Single Agent / Multi-Agent) and model selector (gpt-4o-mini / gpt-4o).
- **Conversation history**: full display of user and assistant messages, with metadata captions showing architecture, model, tools called, elapsed time, and average confidence.
- **Conversational memory**: the last 6 messages are prepended to each new question, enabling 3-turn follow-up resolution.
- **Clear conversation**: resets `st.session_state` and reruns the app.

The app imports directly from `agents.py`, which mirrors the notebook code with adjustments for absolute paths (resolved via `__file__`), explicit `model` parameters (no global `ACTIVE_MODEL`), and `verbose=False` defaults for clean Streamlit output.

IX. Conclusion

The project demonstrates that tool access is the single most important factor for financial Q&A: the baseline's 11% accuracy vs. the single agent's 75% is entirely explained by the ability to retrieve live data. The multi-agent architecture provides a further 8-point improvement (83% overall with gpt-4o-mini) by decomposing complex cross-domain questions into specialist sub-tasks with evidence verification, though it adds latency and offers no benefit on simple single-domain queries.

Surprisingly, the larger gpt-4o model did not outperform gpt-4o-mini in our evaluation, suggesting that for structured tool-calling tasks, the smaller model is cost-effective. The primary architectural weakness is the router's dependency detection: when it fails to recognise a staged execution requirement, specialists lack the context they need, causing cascading failures on hard questions. Future work could address this with a more explicit planning step that constructs a dependency graph before dispatching agents.